

Contents

1	Dynamic Rupture Implementation Plan v3	2
1.1	Changes from v2	2
1.2	Overview	3
1.3	Constraints	3
1.4	Reuse Map	3
2	PART I: MATHEMATICAL FOUNDATIONS	5
2.1	1. Governing Equations (Strong Form)	5
2.1.1	1.1 Velocity-Stress System	5
2.1.2	1.2 Jacobian Matrices	5
2.1.3	1.3 Eigenstructure	6
2.1.4	1.4 Rotation to Face-Normal Coordinates	6
2.2	2. DG Weak Form Derivation	7
2.2.1	2.1 Semi-Discrete Formulation	7
2.2.2	2.2 Integration by Parts	7
2.2.3	2.3 Matrix Form	8
2.2.4	2.4 Mapping to Code	8
2.3	3. Godunov Upwind Flux	9
2.3.1	3.1 Interior Faces	9
2.3.2	3.2 Absorbing Boundary Conditions	9
2.3.3	3.3 Free Surface Boundary Conditions	10
2.4	4. Fault Riemann Solver	10
2.4.1	4.1 Trial Traction (Locked-Fault Godunov State)	10
2.4.2	4.2 Friction Solve	11
2.4.3	4.3 Slip Rate Decomposition	12
2.4.4	4.4 Corrected Traction	12
2.4.5	4.5 Imposed State (Uphoff Eq. 4.60)	12
2.4.6	4.6 State Variable Update	13
2.4.7	4.7 CFL Condition	13
3	PART II: IMPLEMENTATION PHASES	13
3.1	Phase 1: WaveOperator Core — Volume Integrals + Interior Flux	13
3.1.1	Goal	13
3.1.2	Files to Create	13
3.1.3	Files to Modify	13
3.1.4	Detailed Requirements	14
3.1.5	Unit Tests for Phase 1	17
3.1.6	Acceptance Criteria	18
3.1.7	Dependencies	18
3.2	Phase 2: Boundary Conditions — Absorbing + Free Surface	18
3.2.1	Goal	18
3.2.2	Files to Modify	18
3.2.3	Files to Create	18
3.2.4	Detailed Requirements	18
3.2.5	Unit Tests for Phase 2	19
3.2.6	Acceptance Criteria	20
3.2.7	Dependencies	20
3.3	Phase 3: Fault-Face Riemann Solver Using Existing Friction Infrastructure	20
3.3.1	Goal	20
3.3.2	Files to Create	20
3.3.3	Files to Modify	20
3.3.4	Detailed Requirements	20

3.3.5	Unit Tests for Phase 3	22
3.3.6	Acceptance Criteria	22
3.3.7	Dependencies	22
3.4	Phase 4: TPV102 Full Benchmark	23
3.4.1	Goal	23
3.4.2	TPV102 Parameters	23
3.4.3	Friction Law	23
3.4.4	Nucleation	23
3.4.5	Initial θ from Self-Consistency	23
3.4.6	Files to Create	24
3.4.7	Unit Tests for Phase 4	24
3.4.8	Acceptance Criteria	24
3.4.9	Dependencies	24
3.5	Phase 5: BP5 QD→Dynamic Transfer	24
3.5.1	Goal	24
3.5.2	Activation/Deactivation Criteria	25
3.5.3	State Conversion: QD → FD	25
3.5.4	State Conversion: FD → QD	25
3.5.5	Files to Create	25
3.5.6	Unit Tests for Phase 5	25
3.5.7	Acceptance Criteria	25
3.5.8	Dependencies	26
3.6	Phase 6: GPU Acceleration	26
3.6.1	Goal	26
3.6.2	MFEM GPU Infrastructure	26
3.6.3	Task Breakdown	26
3.6.4	GPU Volume Integral Kernel (Task 6.3)	26
3.6.5	GPU Fault Friction Kernel (Task 6.6)	27
3.6.6	Files to Create	27
3.6.7	Unit Tests for Phase 6	27
3.6.8	Acceptance Criteria	27
3.6.9	Dependencies	27
4	PART III: CROSS-CUTTING CONCERNS	27
4.1	Testing Strategy Summary	27
4.2	Risk Assessment	28
4.3	Project Layout After All Phases	28

1 Dynamic Rupture Implementation Plan v3

Date: 2026-04-12 **Status:** Draft — full derivations, unit tests, SeisSol-aligned algorithms **Predecessor:** v2 (lacked derivations, unit test design, absorbing BC equations, GPU specifics) **Benchmark:** SCEC TPV102 (half-space, aging law rate-and-state friction) **References:**

- Dumbser & Käser (2006), ADER-DG for 3D elastic waves
- de la Puente et al. (2009), fault Riemann solver for dynamic rupture
- Pelties et al. (2014), SeisSol fault coupling (eq. A2)
- Uphoff (2020), PhD dissertation, Chapter 4 (eq. 4.51–4.60)
- SeisSol source code /Users/chunhuizhao/projects/SeisSol
- SCEC TPV101/102 benchmark specification (SCEC_validation_ageing_law.pdf)

1.1 Changes from v2

#	v2 Gap	v3 Resolution
1	No unit tests designed in plan	Every new function has a named unit test with inputs, expected outputs, and tolerance
2	Absorbing BC: no equation or algorithm	Full derivation from eigendecomposition; SeisSol getTransposedGodunovState verified
3	No strong→weak→matrix derivation	Complete derivation: PDE → IBP → element matrices → code mapping
4	TPV102: no SeisSol mesh reference	Use SeisSol-compatible tetrahedral mesh; match geometry, stations, output format
5	Phase 5 was generic QD↔FD transfer	Specific BP5 QD→dynamic transfer: trigger criteria, state mapping, time-stepper handoff
6	GPU phase was vague	Kernel-by-kernel plan with MFEM MFEM_FORALL patterns, memory layout, profiling targets

1.2 Overview

Build a 3D dynamic rupture DG code by extending the existing SEAS-MFEM module hierarchy. The `WaveOperator` inherits from `DomainOperator<MeshType>` and implements `TimeDependentOperator::Mult()`, reusing `FaultBasis`, `FaultGeometry`, `DieterichRuinaFriction`, `BoundaryConfig`, `ConstitutiveModel`, and `ProbeOutput` unchanged. New code is limited to: (1) velocity-stress DG volume/face assembly, (2) Godunov flux derived from eigen-decomposition of the Jacobian, (3) fault-face trial traction and imposed state arithmetic, (4) TPV102-specific initialization.

Estimated new code: ~4,000 LOC (increased from v2 due to comprehensive unit tests).

1.3 Constraints

- **Interface constraints:** `DomainOperator`, `ConstitutiveModel`, `FaultBasis`, `FaultGeometry`, `DieterichRuinaFriction` interfaces unchanged. `WaveOperator` adds to them, never modifies.
- **Dependency constraints:** No new external libraries. Uses MFEM `L2_FECollection`, `ODESolver`, `ParMesh`.
- **Convention constraints:** Follow module-per-directory (`dynamic/`), test-per-feature (`test_wave_*.cpp`), TOML config, Makefile integration.
- **Numerical constraints:** CFL-limited explicit stepping. Brent solver convergence $|g| < 10^{-8}$. Energy conservation at fault interface.

1.4 Reuse Map

Existing Component	File	Reuse for Dynamic	How
<code>DomainOperator<MeshType></code>	<code>domain/domain_Base.cpp</code>	Base class for <code>WaveOperator</code>	Inherit; implement <code>Solve()</code> , <code>ComputeTraction()</code> , fault accessors

Existing Component	File	Reuse for Dynamic	How
ConstitutiveModel / LinearElastic	constitutive/* Material	Material properties, wave speeds, stiffness tensor	ComputeTangent() returns 6×6 Voigt $\rightarrow$ derive A, B, C Jacobians
BoundaryConfig	domain/boundary/* Boundary	Absorbing + free-surface BC classification	natural_attrs $\rightarrow$ free surface; add absorbing_attrs field
DomainConfig	domain/domain_* Solver	Solver parameters (face_basis_type, penalty)	Add cfl_factor field for explicit stepping
FaultBasis	fault/fault_basis/* Normal	Normal/tangent transforms at fault quad points	Direct call: ProjectTraction(), NormalStress(), EmbedSlipQP()
FaultGeometry	fault/fault_geometry/* Fault	Fault DOF management, spatial params	Construct from WaveOperator via DomainOperator<MeshType>&
DieterichRuinaFriction	friction/dieterich/* Friction	Friction solver $(\tau, \psi, \sigma_n, \eta, a) \rightarrow V$	SolveSlipRateVectorPsi() with Riemann trial traction as τ
AgingLawPsi	friction/state/* StateVariable	State variable evolution $d\psi/dt$	Evaluate(V, psi, ...) for RK integration of ψ alongside Q
SEASConfig	config/seas_config/* Config	Configuration + TOML parsing	Extend SimulationConfig with DynamicSettings
ProbeOutput	io/probe_output/* Station	Station time series output	Direct use — already generic columnar I/O

2 PART I: MATHEMATICAL FOUNDATIONS

2.1 1. Governing Equations (Strong Form)

2.1.1 1.1 Velocity-Stress System

The 3D isotropic linear elastic wave equation in first-order velocity-stress form.

State vector (9 unknowns):

$$\mathbf{Q} = \begin{pmatrix} \sigma_{xx} \\ \sigma_{yy} \\ \sigma_{zz} \\ \sigma_{xy} \\ \sigma_{yz} \\ \sigma_{xz} \\ v_x \\ v_y \\ v_z \end{pmatrix}$$

Index convention: [SXX=0, SYY=1, SZZ=2, SXY=3, SYZ=4, SXZ=5, VX=6, VY=7, VZ=8]

Conservation law:

$$\frac{\partial \mathbf{Q}}{\partial t} + \mathbf{A} \frac{\partial \mathbf{Q}}{\partial x} + \mathbf{B} \frac{\partial \mathbf{Q}}{\partial y} + \mathbf{C} \frac{\partial \mathbf{Q}}{\partial z} = \mathbf{0} \quad (1)$$

where $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$ are 9×9 Jacobian (flux) matrices.

2.1.2 1.2 Jacobian Matrices

Derived from Hooke's law ($\sigma_{ij} = \lambda \delta_{ij} \varepsilon_{kk} + 2\mu \varepsilon_{ij}$) and momentum conservation ($\rho \partial v_i / \partial t = \partial \sigma_{ij} / \partial x_j$).

A-matrix (x-direction flux), from SeisSol getTransposedCoefficientMatrix() (ElasticSetup.h:28-77):

$$\mathbf{A} = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & -(\lambda + 2\mu) & -\lambda & -\lambda \\ 0 & 0 & 0 & 0 & 0 & 0 & -\lambda & -(\lambda + 2\mu) & -\lambda \\ 0 & 0 & 0 & 0 & 0 & 0 & -\lambda & -\lambda & -(\lambda + 2\mu) \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -\mu & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -\mu \\ 0 & 0 & 0 & 0 & 0 & 0 & -\mu & 0 & 0 \\ -\frac{1}{\rho} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -\frac{1}{\rho} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -\frac{1}{\rho} & 0 & 0 & 0 \end{pmatrix}$$

Physical meaning: $\mathbf{A} \partial \mathbf{Q} / \partial x$ couples:

- $\partial v_x / \partial x \rightarrow \partial \sigma_{xx} / \partial t$ via $-(\lambda + 2\mu)$ (P-wave compression)
- $\partial v_y / \partial x \rightarrow \partial \sigma_{xy} / \partial t$ via $-\mu$ (S-wave shear)
- $\partial \sigma_{xx} / \partial x \rightarrow \partial v_x / \partial t$ via $-1/\rho$ (Newton's 2nd law)

B-matrix (y-direction): Obtained from $\mathbf{A}$ by cyclic permutation $x \leftrightarrow y$:

$$B_{7,0} = -\lambda, \quad B_{7,1} = -(\lambda + 2\mu), \quad B_{7,2} = -\lambda$$

$$B_{6,3} = -\mu, \quad B_{8,4} = -\mu$$

$$B_{1,7} = -\frac{1}{\rho}, \quad B_{3,6} = -\frac{1}{\rho}, \quad B_{4,8} = -\frac{1}{\rho}$$

C-matrix (z-direction): Cyclic permutation $x \rightarrow y \rightarrow z$:

$$C_{8,0} = -\lambda, \quad C_{8,1} = -\lambda, \quad C_{8,2} = -(\lambda + 2\mu)$$

$$C_{7,4} = -\mu, \quad C_{6,5} = -\mu$$

$$C_{2,8} = -\frac{1}{\rho}, \quad C_{5,6} = -\frac{1}{\rho}, \quad C_{4,7} = -\frac{1}{\rho}$$

2.1.3 1.3 Eigenstructure

The normal-direction Jacobian $\mathbf{A}_n = n_x \mathbf{A} + n_y \mathbf{B} + n_z \mathbf{C}$ has eigenvalues:

$$= \text{diag}(+c_p, +c_s, +c_s, 0, 0, 0, -c_s, -c_s, -c_p)$$

where:

$$c_p = \sqrt{\frac{\lambda + 2\mu}{\rho}} \quad (\text{P-wave}), \quad c_s = \sqrt{\frac{\mu}{\rho}} \quad (\text{S-wave})$$

Eigenvector matrix R (from SeisSol ElasticSetup.h:88-139).

For face with normal $\mathbf{n} = (1, 0, 0)$, tangent $\mathbf{t}_1 = (0, 1, 0)$, tangent $\mathbf{t}_2 = (0, 0, 1)$:

$$\mathbf{R} = \begin{pmatrix} \lambda+2\mu & 0 & 0 & \cdots & 0 & 0 & \lambda+2\mu \\ \lambda & 0 & 0 & \cdots & 0 & 0 & \lambda \\ \lambda & 0 & 0 & \cdots & 0 & 0 & \lambda \\ 0 & \mu & 0 & \cdots & 0 & \mu & 0 \\ 0 & 0 & 0 & \cdots & 0 & 0 & 0 \\ 0 & 0 & \mu & \cdots & \mu & 0 & 0 \\ +\sqrt{Z_p/\rho} & 0 & 0 & \cdots & 0 & 0 & -\sqrt{Z_p/\rho} \\ 0 & +\sqrt{\mu/\rho} & 0 & \cdots & 0 & -\sqrt{\mu/\rho} & 0 \\ 0 & 0 & +\sqrt{\mu/\rho} & \cdots & -\sqrt{\mu/\rho} & 0 & 0 \end{pmatrix}$$

where $Z_p = \rho c_p$ (P-impedance) and $Z_s = \rho c_s$ (S-impedance).

2.1.4 1.4 Rotation to Face-Normal Coordinates

For an arbitrary face with unit normal $\mathbf{n}$, tangent $\mathbf{t}_1$, tangent $\mathbf{t}_2$, the state $\mathbf{Q}$ is rotated via a 9×9 block-diagonal rotation matrix:

$$\mathbf{T} = \begin{pmatrix} \mathbf{T}_\sigma & \mathbf{0} \\ \mathbf{0} & \mathbf{T}_v \end{pmatrix}_{9 \times 9}$$

Stress rotation $\mathbf{T}_\sigma$ (6×6) from SeisSol symmetricTensor2RotationMatrix() (Transformation.cpp:128-181):

$$(\mathbf{T}_\sigma)_{ij} = \begin{pmatrix} n_x^2 & t_{1x}^2 & t_{2x}^2 & 2n_x t_{1x} & 2t_{1x} t_{2x} & 2n_x t_{2x} \\ n_y^2 & t_{1y}^2 & t_{2y}^2 & 2n_y t_{1y} & 2t_{1y} t_{2y} & 2n_y t_{2y} \\ n_z^2 & t_{1z}^2 & t_{2z}^2 & 2n_z t_{1z} & 2t_{1z} t_{2z} & 2n_z t_{2z} \\ n_x n_y & t_{1x} t_{1y} & t_{2x} t_{2y} & n_y t_{1x} + n_x t_{1y} & \dots & \dots \\ n_y n_z & t_{1y} t_{1z} & t_{2y} t_{2z} & \dots & \dots & \dots \\ n_x n_z & t_{1x} t_{1z} & t_{2x} t_{2z} & \dots & \dots & \dots \end{pmatrix}$$

Velocity rotation $\mathbf{T}_v$ (3×3) from SeisSol `tensor1RotationMatrix()` (`Transformation.cpp:101-126`):

$$\mathbf{T}_v = \begin{pmatrix} n_x & t_{1x} & t_{2x} \\ n_y & t_{1y} & t_{2y} \\ n_z & t_{1z} & t_{2z} \end{pmatrix}$$

In rotated coordinates, the normal-direction Jacobian is diagonal in eigenvalue structure, enabling the Godunov flux split.

2.2 DG Weak Form Derivation

2.2.1 2.1 Semi-Discrete Formulation

Partition the domain Ω into non-overlapping elements $\{T_m\}$. In each element, expand:

$$\mathbf{Q}_h(\mathbf{x}, t) = \sum_l \hat{\mathbf{Q}}_l(t) \Phi_l(\mathbf{x}) \quad (2a)$$

where Φ_l are polynomial basis functions of degree N (from `L2_FECollection(order, 3)`).

Multiply Eq. (1) by test function Φ_k and integrate over element T_m :

$$\int_{T_m} \Phi_k \frac{\partial \mathbf{Q}}{\partial t} dV + \int_{T_m} \Phi_k \left(\mathbf{A} \frac{\partial \mathbf{Q}}{\partial x} + \mathbf{B} \frac{\partial \mathbf{Q}}{\partial y} + \mathbf{C} \frac{\partial \mathbf{Q}}{\partial z} \right) dV = 0$$

2.2.2 2.2 Integration by Parts

Apply the divergence theorem to the flux terms. Define the physical flux:

$$\mathbf{F}_j(\mathbf{Q}) = \begin{cases} \mathbf{A}\mathbf{Q} & j = x \\ \mathbf{B}\mathbf{Q} & j = y \\ \mathbf{C}\mathbf{Q} & j = z \end{cases}$$

Integration by parts yields the **DG weak form**:

$$\int_{T_m} \Phi_k \frac{\partial \mathbf{Q}}{\partial t} dV + \oint_{\partial T_m} \Phi_k \mathbf{F}_n^h dS - \int_{T_m} \frac{\partial \Phi_k}{\partial x_j} \mathbf{F}_j(\mathbf{Q}) dV = 0 \quad (2)$$

where $\mathbf{F}_n^h$ is the **numerical flux** at element interfaces, replacing the non-unique trace of the discontinuous solution.

2.2.3 2.3 Matrix Form

Define element matrices:

Mass matrix (block-diagonal for DG — each element independent):

$$M_{kl} = \int_{T_m} \Phi_k \Phi_l dV \quad (n_{\text{dof}} \times n_{\text{dof}} \text{ per component})$$

For vdim=9: total mass matrix is $9n_{\text{dof}} \times 9n_{\text{dof}}$, block-diagonal with 9 identical $n_{\text{dof}} \times n_{\text{dof}}$ blocks.

Stiffness matrices (volume integral, three per element):

$$S_{kl}^x = \int_{T_m} \frac{\partial \Phi_k}{\partial x} \Phi_l dV, \quad S_{kl}^y = \int_{T_m} \frac{\partial \Phi_k}{\partial y} \Phi_l dV, \quad S_{kl}^z = \int_{T_m} \frac{\partial \Phi_k}{\partial z} \Phi_l dV$$

Volume contribution (per element, component q):

$$\text{Vol}_k^q = \sum_l \sum_{j \in \{x,y,z\}} S_{kl}^j [\mathbf{A}_j]_{q,r} \hat{Q}_l^r \quad (2b)$$

Face flux contribution (per face):

$$\text{Face}_k = \oint_{\partial T_m \cap f} \Phi_k \mathbf{F}_n^h dS$$

Semi-discrete ODE:

$$\mathbf{M} \frac{d\hat{\mathbf{Q}}}{dt} = -\text{Face} + \text{Vol} \quad (\text{per element})$$

$$\Rightarrow \frac{d\hat{\mathbf{Q}}}{dt} = \mathbf{M}^{-1}(-\text{Face} + \text{Vol}) \quad (3)$$

Since $\mathbf{M}$ is block-diagonal (DG), the inverse is element-local. For order N in 3D, each element has $n_{\text{dof}} = \frac{(N+1)(N+2)(N+3)}{6}$ DOFs per component (tetrahedra) or $(N+1)^3$ (hexahedra).

2.2.4 2.4 Mapping to Code

Mathematical Object	Code Location	MFEM API
$\Phi_l, \partial \Phi_l / \partial x_j$	fe->CalcShape(), fe->CalcDShape()	FiniteElement
M_{kl}	MassIntegrator or manual assembly	BilinearFormIntegrator
S_{kl}^j	Manual: CalcDShape() contracted with Jacobian	Element-local loop
$\mathbf{A}, \mathbf{B}, \mathbf{C}$	GodunovFlux constructor	Derived from λ, μ, ρ
$\mathbf{T}, \mathbf{T}^{-1}$	GodunovFlux::BuildRotation()	From face normal + tangents
$\mathbf{F}_n^h$ numerical flux	GodunovFlux::Interior/Absorbing/PeriodicDispatch	PerFaceDispatch

Mathematical Object	Code Location	MFEM API
$\mathbf{M}^{-1}$	<code>WaveOperator::elem_mass_inv_[]</code>	Precomputed DenseMatrix per element
$d\mathbf{Q}/dt = \mathbf{M}^{-1}\text{RHS}$	<code>WaveOperator::Mult()</code>	TimeDependentOperator interface

2.3 3. Godunov Upwind Flux

2.3.1 3.1 Interior Faces

At each interior face with outward normal $\mathbf{n}$ (from element “self” to element “nbr”), the Godunov flux uses the eigendecomposition of the normal Jacobian:

$$\mathbf{A}_n = n_x \mathbf{A} + n_y \mathbf{B} + n_z \mathbf{C} = \mathbf{R} \mathbf{R}^{-1}$$

Split into positive and negative eigenvalue contributions:

$$\mathbf{A}_n^+ = \mathbf{R}^+ \mathbf{R}^{-1}, \quad \mathbf{A}_n^- = \mathbf{R}^- \mathbf{R}^{-1}$$

where $^+ = \text{diag}(\max(\lambda_i, 0))$ and $^- = \text{diag}(\min(\lambda_i, 0))$.

The **upwind (Godunov) flux** is:

$$\mathbf{F}_n^h = \mathbf{A}_n^+ \mathbf{Q}_{\text{self}} + \mathbf{A}_n^- \mathbf{Q}_{\text{nbr}} = \frac{1}{2}(\mathbf{A}_n + |\mathbf{A}_n|) \mathbf{Q}_{\text{self}} + \frac{1}{2}(\mathbf{A}_n - |\mathbf{A}_n|) \mathbf{Q}_{\text{nbr}} \quad (4)$$

where $|\mathbf{A}_n| = \mathbf{R} || \mathbf{R}^{-1}$.

Implementation via rotation (avoids 9×9 eigendecomposition per face):

1. Rotate to face-aligned coordinates: $\mathbf{Q}_{\text{rot}} = \mathbf{T}^{-1} \mathbf{Q}$
2. In rotated frame, $\mathbf{A}_n$ becomes the x-direction Jacobian (diagonal eigenstructure known analytically)
3. Apply pre-computed split matrices $\mathbf{A}_x^+$, $\mathbf{A}_x^-$
4. Rotate back: $\mathbf{F} = \mathbf{T} \mathbf{F}_{\text{rot}}$

For **homogeneous material**, $\mathbf{A}_x^+$ and $\mathbf{A}_x^-$ are constant — precompute once at construction.

SeisSol verification: In `ElasticSetup.h`:146–165:

$$\chi = \text{diag}(1, 1, 1, 0, 0, 0, 0, 0, 0) \quad (\text{selects 3 incoming characteristics})$$

$$\text{godunov} = \mathbf{R} \chi \mathbf{R}^{-1}, \quad \text{qGodLocal} = \mathbf{I} - \text{godunov}$$

2.3.2 3.2 Absorbing Boundary Conditions

Equation (from SeisSol and Dumbser & Käser 2006):

$$\mathbf{F}_n^{\text{abs}} = \mathbf{A}_n^+ \mathbf{Q}_{\text{self}} \quad (5)$$

Physical meaning: Only outgoing waves (positive eigenvalues w.r.t. outward normal) contribute. Incoming waves are set to zero — equivalent to an infinite domain with no reflected energy.

Derivation: At an absorbing boundary, the ghost state $\mathbf{Q}_{\text{ghost}}$ satisfies $\mathbf{A}_n^- \mathbf{Q}_{\text{ghost}} = \mathbf{0}$. Since $\mathbf{A}_n^-$ projects onto incoming characteristics, this means no incoming waves:

$$\mathbf{F}_n^h = \mathbf{A}_n^+ \mathbf{Q}_{\text{self}} + \underbrace{\mathbf{A}_n^- \mathbf{Q}_{\text{ghost}}}_{=0} = \mathbf{A}_n^+ \mathbf{Q}_{\text{self}}$$

Limitation: First-order absorbing BC — perfect only at normal incidence. At oblique incidence angle θ , the reflection coefficient is $O(\sin^2 \theta)$. For TPV102, the domain is large enough that boundary reflections don't contaminate the fault region during the 12 s simulation.

2.3.3 3.3 Free Surface Boundary Conditions

Equation: Zero traction at the free surface: $\sigma \cdot \mathbf{n} = \mathbf{0}$.

The ghost state mirrors the stress components that produce traction on the surface:

$$\mathbf{Q}_{\text{ghost}} = \Gamma \mathbf{Q}_{\text{self}}$$

where Γ is the **stress mirror matrix**. In the face-normal coordinate system ($\mathbf{n}$ aligned with first axis):

$$\Gamma = \text{diag} \left(\underbrace{-1}_{\sigma_{nn}}, \underbrace{+1}_{\sigma_{t_1 t_1}}, \underbrace{+1}_{\sigma_{t_2 t_2}}, \underbrace{-1}_{\sigma_{nt_1}}, \underbrace{+1}_{\sigma_{t_1 t_2}}, \underbrace{-1}_{\sigma_{nt_2}}, \underbrace{+1}_{v_n}, \underbrace{+1}_{v_{t_1}}, \underbrace{+1}_{v_{t_2}} \right)$$

Rule: Any stress component with an odd number of normal indices flips sign. This ensures $\sigma \cdot \mathbf{n} = \mathbf{0}$ at the free surface.

The flux becomes:

$$\boxed{\mathbf{F}_n^{\text{free}} = \mathbf{A}_n^+ \mathbf{Q}_{\text{self}} + \mathbf{A}_n^- \Gamma \mathbf{Q}_{\text{self}}} \quad (6)$$

SeisSol derivation: In `getTransposedFreeSurfaceGodunovState()` (Model/Common.h:251–296):

$$\mathbf{S} = -\mathbf{R}_{21} \mathbf{R}_{11}^{-1}$$

where $\mathbf{R}_{11}$ is the stress block of the incoming eigenvectors and $\mathbf{R}_{21}$ is the velocity block. This produces the same result as the Γ mirror approach — both enforce zero-traction at the boundary.

2.4 4. Fault Riemann Solver

2.4.1 4.1 Trial Traction (Locked-Fault Godunov State)

Following Uphoff (2020) Eq. 4.51 and `SeisSol precomputeStressFromQInterpolated()` (FrictionSolverCommon.h:143–194).

Impedances:

$$Z_p = \rho c_p, \quad Z_s = \rho c_s$$

$$\eta_p = \frac{Z_p^+ Z_p^-}{Z_p^+ + Z_p^-} \quad (\text{harmonic mean}), \quad \eta_s = \frac{Z_s^+ Z_s^-}{Z_s^+ + Z_s^-}$$

For homogeneous material: $\eta_p = Z_p/2$, $\eta_s = Z_s/2$.

Trial traction (assuming locked fault — zero slip rate):

$$\sigma_n^{\text{trial}} = \eta_p \left(v_n^- - v_n^+ + \frac{\sigma_n^+}{Z_p^+} + \frac{\sigma_n^-}{Z_p^-} \right) \quad (7a)$$

$$\tau_1^{\text{trial}} = \eta_s \left(v_{t_1}^- - v_{t_1}^+ + \frac{\tau_1^+}{Z_s^+} + \frac{\tau_1^-}{Z_s^-} \right) \quad (7b)$$

$$\tau_2^{\text{trial}} = \eta_s \left(v_{t_2}^- - v_{t_2}^+ + \frac{\tau_2^+}{Z_s^+} + \frac{\tau_2^-}{Z_s^-} \right) \quad (7c)$$

where subscripts n, t_1, t_2 denote components in the fault-local basis (normal, dip-tangent, strike-tangent).

SeisSol code mapping: $U = v_n, V = v_{t_1}, W = v_{t_2}, N = \sigma_n, T1 = \tau_1, T2 = \tau_2$.

2.4.2 4.2 Friction Solve

Total traction (initial + trial):

$$\sigma_n^{\text{total}} = \sigma_{n,0} + \sigma_n^{\text{trial}}$$

$$\Theta = \sqrt{(\tau_{1,0} + \tau_1^{\text{trial}})^2 + (\tau_{2,0} + \tau_2^{\text{trial}})^2}$$

where $\sigma_{n,0}, \tau_{1,0}, \tau_{2,0}$ are the pre-stress (initial conditions + nucleation perturbation).

Friction law equation (same form as QD — the **key reuse insight**):

$$\Theta = |\sigma_n^{\text{total}}| f(\hat{V}, \psi) + \eta_s \hat{V} \quad (8)$$

This is **identical** to the QD balance where:

- **QD:** $\eta = \mu/(2c_s) = Z_s/2 = \eta_s$ (for homogeneous material)
- **DR:** $\eta_s = Z_s/2$ (harmonic mean impedance)

The existing `SolveSlipRateVectorPsi` solves Eq. (8) for $\hat{V}$ using Brent's method in $\log_{10}(V)$ space, handling all edge cases documented in debug v1-v13.

SeisSol uses Newton-Raphson (`RateAndState.h:305–341`):

$$g(\hat{V}) = -\frac{1}{\eta_s} (|\sigma_n| \mu(\hat{V}, \psi) - \Theta) - \hat{V} = 0$$

$$\frac{dg}{d\hat{V}} = -\frac{1}{\eta_s} |\sigma_n| \frac{d\mu}{d\hat{V}} - 1$$

$$\hat{V}_{i+1} = \max\left(10^{-45}, \hat{V}_i - \frac{g}{dg/d\hat{V}}\right)$$

We use Brent's method instead — proven more robust for extreme ψ values (see debug v1, v7). This is a deliberate, well-justified deviation from SeisSol.

2.4.3 4.3 Slip Rate Decomposition

$$\text{strength} = |\sigma_n^{\text{total}}| f(\hat{V}, \psi)$$

$$V_1 = \hat{V} \frac{\tau_{1,0} + \tau_1^{\text{trial}}}{\text{strength} + \eta_s \hat{V}}, \quad V_2 = \hat{V} \frac{\tau_{2,0} + \tau_2^{\text{trial}}}{\text{strength} + \eta_s \hat{V}} \quad (9)$$

The direction of slip is aligned with the total traction vector.

2.4.4 4.4 Corrected Traction

$$\tau_1^{\text{corr}} = \tau_1^{\text{trial}} - \eta_s V_1, \quad \tau_2^{\text{corr}} = \tau_2^{\text{trial}} - \eta_s V_2 \quad (10)$$

$$\sigma_n^{\text{corr}} = \sigma_n^{\text{trial}} \quad (\text{unchanged by friction — slip is tangential only})$$

2.4.5 4.5 Imposed State (Uphoff Eq. 4.60)

From SeisSol `postcomputeImposedStateFromNewStress()` (FrictionSolverCommon.h:288–365):

Minus side (Q^-):

$$v_n^{-,\text{imp}} = v_n^- - \frac{1}{Z_p^-} (\sigma_n^{\text{corr}} - \sigma_n^-) \quad (11a)$$

$$v_{t_1}^{-,\text{imp}} = v_{t_1}^- - \frac{1}{Z_s^-} (\tau_1^{\text{corr}} - \tau_1^-) \quad (11b)$$

$$v_{t_2}^{-,\text{imp}} = v_{t_2}^- - \frac{1}{Z_s^-} (\tau_2^{\text{corr}} - \tau_2^-) \quad (11c)$$

$$\sigma_n^{-,\text{imp}} = \sigma_n^{\text{corr}}, \quad \tau_1^{-,\text{imp}} = \tau_1^{\text{corr}}, \quad \tau_2^{-,\text{imp}} = \tau_2^{\text{corr}} \quad (11d)$$

Plus side (Q^+):

$$v_n^{+,\text{imp}} = v_n^+ + \frac{1}{Z_p^+} (\sigma_n^{\text{corr}} - \sigma_n^+) \quad (12a)$$

$$v_{t_1}^{+,\text{imp}} = v_{t_1}^+ + \frac{1}{Z_s^+} (\tau_1^{\text{corr}} - \tau_1^+) \quad (12b)$$

$$v_{t_2}^{+,\text{imp}} = v_{t_2}^+ + \frac{1}{Z_s^+} (\tau_2^{\text{corr}} - \tau_2^+) \quad (12c)$$

$$\sigma_n^{+,\text{imp}} = \sigma_n^{\text{corr}}, \quad \tau_1^{+,\text{imp}} = \tau_1^{\text{corr}}, \quad \tau_2^{+,\text{imp}} = \tau_2^{\text{corr}} \quad (12d)$$

Physical meaning: The imposed velocities ensure the Godunov state is consistent with the friction-limited traction. Stress is continuous across the fault (traction continuity), but velocity has a jump equal to the slip rate.

2.4.6 4.6 State Variable Update

Aging law in ψ -space (from SeisSol AgingLaw.h:39–49):

$$\frac{d\psi}{dt} = 1 - \frac{V\psi}{L}$$

Analytic solution over time step Δt :

$$\psi(t + \Delta t) = \psi(t) e^{-V\Delta t/L} + \frac{L}{V}(1 - e^{-V\Delta t/L}) \quad (13)$$

Using `expm1()` for numerical stability when $V\Delta t/L \ll 1$:

$$\psi_{\text{new}} = \psi_{\text{old}} e^{-V\Delta t/L} + \frac{L}{V}(-\text{expm1}(-V\Delta t/L))$$

This matches SeisSol exactly: `stateVarReference * exp1v + localSl0 / localSlipRate * exp1m`.

2.4.7 4.7 CFL Condition

$$\Delta t \leq \text{CFL} \frac{h_{\min}}{c_p}, \quad \text{CFL} \sim \frac{1}{2N + 1} \quad (14)$$

where $h_{\min}$ is the minimum element diameter (inscribed sphere for tetrahedra) and N is the polynomial order.

3 PART II: IMPLEMENTATION PHASES

3.1 Phase 1: WaveOperator Core — Volume Integrals + Interior Flux

3.1.1 Goal

A `WaveOperator<ParMesh>` that inherits `DomainOperator<ParMesh>`, solves the velocity-stress wave equation on a homogeneous cube with Godunov flux at interior faces, verified by plane wave convergence.

3.1.2 Files to Create

- `dynamic/wave_state.hpp` — `Q` index enum, state vector utilities
- `dynamic/wave_operator.hpp` — Class declaration inheriting `DomainOperator<MeshType> + TimeDependentOperator`
- `dynamic/wave_operator.cpp` — Volume integral, face flux, mass inverse
- `dynamic/godunov_flux.hpp` — Upwind flux: rotation, split, absorbing, free surface
- `dynamic/godunov_flux.cpp` — Implementation (~400 LOC)
- `tests/unit/test_godunov_flux.cpp` — Flux unit tests (8 tests)
- `tests/unit/test_wave_operator.cpp` — Operator integration tests (6 tests)

3.1.3 Files to Modify

- `domain/boundary_config.hpp` — Add `absorbing_attrs` field (one line)
- `domain/domain_config.hpp` — Add `cfl_factor` field (one line)
- `Makefile` — Add dynamic rupture build targets

3.1.4 Detailed Requirements

3.1.4.1 1. WaveState Index Enum

```
// dynamic/wave_state.hpp
namespace seas::dynamic {

enum QIndex : int {
    SXX = 0, SYY = 1, SZZ = 2, // Normal stresses
    SXY = 3, SYZ = 4, SXZ = 5, // Shear stresses
    VX = 6, VY = 7, VZ = 8    // Velocities
};

static constexpr int NUM_STATE = 9;

/// Compute elastic energy density:
///  $E = (1/2) \sigma:\epsilon + (1/2) \rho v.v$ 
real_t ComputeEnergyDensity(const real_t Q[9], real_t lambda, real_t mu, real_t rho);

/// Compute kinetic energy density:  $(1/2) \rho (v_x^2 + v_y^2 + v_z^2)$ 
real_t ComputeKineticEnergy(const real_t Q[9], real_t rho);

/// Compute strain energy from stress
real_t ComputeStrainEnergy(const real_t Q[9], real_t lambda, real_t mu);

} // namespace
```

3.1.4.2 2. GodunovFlux Class

```
// dynamic/godunov_flux.hpp
class GodunovFlux {
public:
    GodunovFlux(real_t lambda, real_t mu, real_t rho);

    /// --- Jacobian access (for unit testing) ---
    void GetJacobian(int dim, DenseMatrix &A_dim) const; // dim=0,1,2

    /// --- Rotation matrices ---
    static void BuildRotation(const real_t normal[3],
                             const real_t tangent1[3],
                             const real_t tangent2[3],
                             DenseMatrix &T, DenseMatrix &Tinv);

    /// --- Split flux matrices ---
    void GetSplitFlux(DenseMatrix &Aplus, DenseMatrix &Aminus) const;

    /// --- Flux evaluation (Eq. 4, 5, 6) ---
    void Interior(const real_t normal[3],
                 const real_t Q_self[9], const real_t Q_nbr[9],
                 real_t flux[9]) const;

    void Absorbing(const real_t normal[3],
                  const real_t Q_self[9],
                  real_t flux[9]) const;

    void FreeSurface(const real_t normal[3],
```

```

        const real_t Q_self[9],
        real_t flux[9]) const;

    real_t MaxWaveSpeed() const { return cp_; }
    real_t GetCp() const { return cp_; }
    real_t GetCs() const { return cs_; }
    real_t GetZp() const { return rho_ * cp_; }
    real_t GetZs() const { return rho_ * cs_; }

private:
    real_t lambda_, mu_, rho_, cp_, cs_;
    DenseMatrix Aplus_, Aminus_; // 9x9, precomputed
};

```

3.1.4.3 3. WaveOperator<MeshType> Class

```

// dynamic/wave_operator.hpp
template <typename MeshType = ParMesh>
class WaveOperator : public DomainOperator<MeshType>,
                    public TimeDependentOperator
{
public:
    WaveOperator(MeshType &mesh, int order,
                const ConstitutiveModel &model,
                real_t rho,
                const BoundaryConfig &bdr_config,
                const DomainConfig &config = {});

    // --- TimeDependentOperator interface ---
    // Implements Eq. (3):  $dQ/dt = M^{-1}(-Face + Vol)$ 
    void Mult(const Vector &Q, Vector &dQdt) const override;

    // --- DomainOperator interface ---
    int NumComponents() const override { return 3; }
    int Dimension() const override { return 3; }
    int NumSlipComponents() const override { return 2; }
    void Solve(real_t time, const Vector &slip_bc,
              GridFuncType &displacement) override;
    void ComputeTraction(const GridFuncType &displacement,
                        const Vector &slip_bc,
                        Vector &traction,
                        Vector *normal_traction = nullptr) override;
    FESpaceType &GetFESpace() override;
    MeshType &GetMesh() override;
    real_t GetShearModulus() const override;
    int GetNumFaultDOFs() const override;
    void GetFaultDepths(Vector &depths) const override;
    const Array<int> &GetFaultDOFs() const override;
    const FaultBasis *GetFaultBasis() const override;

    // --- Wave-specific ---
    real_t GetMaxDt() const; // CFL: Eq. (14)
    real_t GetCp() const;
    real_t GetCs() const;

```

```

const GodunovFlux &GetFlux() const;
void SetFaultFlux(FaultFaceFlux *fault_flux);

private:
void AssembleElementMassInverse();
void ComputeVolumeRHS(const Vector &Q, Vector &rhs) const;    // Eq. (2b)
void ComputeFaceFluxRHS(const Vector &Q, Vector &rhs) const;  // Eq. (4-6)

enum class FaceType { Interior, Absorbing, FreeSurface, Fault };
void ClassifyFaces();
void SetupFaultInfo();

MeshType &mesh_;
int order_;
real_t rho_, lambda_, mu_, cp_, cs_;
GodunovFlux flux_;
BoundaryConfig bdr_config_;
DomainConfig config_;

std::unique_ptr<L2_FECollection> fec_;
std::unique_ptr<FESpaceType> fes_;    // L2, vdim=9

std::vector<DenseMatrix> elem_mass_inv_;
std::vector<FaceType> face_types_;
FaultBasis fault_basis_;
Array<int> fault_dofs_;
Vector fault_depths_;
FaultFaceFlux *fault_flux_ = nullptr;
};

```

3.1.4.4 4. Mult() Implementation (Core Time Step, Eq. 3)

```

template <typename MeshType>
void WaveOperator<MeshType>::Mult(const Vector &Q, Vector &dQdt) const
{
    dQdt = 0.0;
    ComputeVolumeRHS(Q, dQdt);    // Step 1: Volume integral (Eq. 2b)
    ComputeFaceFluxRHS(Q, dQdt);  // Step 2: Face flux (Eq. 4/5/6)
    ApplyMassInverse(dQdt);        // Step 3: M^{-1} (element-local)
}

```

3.1.4.5 5. Volume Integral Implementation Implements the volume term of Eq. (2):

$$\text{rhs}_k^q += \sum_{j \in \{x,y,z\}} w_q \det J \left. \frac{\partial \Phi_k}{\partial x_j} \right|_{\mathbf{x}_q} \cdot [\mathbf{A}_j \mathbf{Q}(\mathbf{x}_q)]^q$$

// For each element e, quadrature point q, component c, DOF i:
rhs_e[c * ndof + i] += w * dshape(i, j) * F[j][c];
// where F[j][c] = (A_j * Q_qp)[c]

3.1.4.6 6. Face Flux Implementation Dispatches per face type:

```

switch (face_types_[f]) {
    case FaceType::Interior:    flux_.Interior(n, Q_self, Q_nbr, F_h);    break; // Eq. 4
}

```



```

case FaceType::Absorbing: flux_.Absorbing(n, Q_self, F_h); break; // Eq. 5
case FaceType::FreeSurface: flux_.FreeSurface(n, Q_self, F_h); break; // Eq. 6
case FaceType::Fault: /* Phase 3 */ break;
}

```

3.1.5 Unit Tests for Phase 1

3.1.5.1 Test File: tests/unit/test_godunov_flux.cpp

#	Test Name	What	How	Tolerance
1	TestJacobianSymmetry	A , B , C have correct nonzero entries	Check $A_{6,0} = -1/\rho$, $A_{0,6} = -(\lambda + 2\mu)$, etc.	Exact
2	TestEigenvalues	Eigenvalues of A_n are $\{\pm c_p, \pm c_s, \pm c_s, 0, 0, 0\}$	5 random normals, eigendecompose	$< 10^{-10}$
3	TestRotationOrthogonality	$\mathbf{T}\mathbf{T}^{-1} = \mathbf{I}$	4 orientations	$\ \mathbf{T}\mathbf{T}^{-1} - \mathbf{I}\ _F < 10^{-14}$
4	TestInteriorFluxConsistency	$\mathbf{Q}_{\text{self}} = \mathbf{Q}_{\text{nbr}} \Rightarrow \mathbf{F} = \mathbf{A}_n \mathbf{Q}$	Random uniform Q	$< 10^{-12}$
5	TestInteriorFluxConservation	$\mathbf{F}(\mathbf{Q}_L, \mathbf{Q}_R; \mathbf{n}) + \mathbf{F}(\mathbf{Q}_R, \mathbf{Q}_L; -\mathbf{n}) = \mathbf{0}$	Random Q_L , Q_R	$< 10^{-12}$
6	TestAbsorbingFluxOutgoingOnly	Outgoing P-wave: full flux; incoming P-wave: zero flux	Construct $\mathbf{Q}_P^\pm$ analytically	$< 10^{-12}$
7	TestFreeSurfaceZeroTraction	Γ mirror produces zero traction	Random Q , check $\sigma^{\text{imp}} \cdot \mathbf{n} = 0$	$< 10^{-10}$
8	TestSplitFluxConsistency	$\mathbf{A}^+ + \mathbf{A}^- = \mathbf{A}_x$	Compare matrices	$< 10^{-14}$

3.1.5.2 Test File: tests/unit/test_wave_operator.cpp

#	Test Name	What	How	Tolerance
9	TestMassMatrixInverse	$\mathbf{M}\mathbf{M}^{-1} = \mathbf{I}$ per element	2x2x2 hex mesh, order 2	$< 10^{-12}$
10	TestPlaneWavePSpeed	P-wave speed = c_p	Advance one period, measure phase error	$ c/c_p - 1 < 0.01$
11	TestPlaneWaveSSpeed	S-wave speed = c_s	Same as Test 10 for S-wave	$ c/c_s - 1 < 0.01$
12	TestConvergenceOrder	L^2 error $\propto h^{N+1}$	$h = 1/4, 1/8, 1/16$; $N = 1, 2$	Rate $\geq N + 0.5$

#	Test Name	What	How	Tolerance
13	TestEnergyConservation	$\ E_f/E_0 - 1\ < \epsilon$ in reflecting box	4×4×4, all free surface, 1000 steps	$< 10^{-10}$
14	TestFaultGeometryConstruction	FaultGeometry(waveMesh ...) constructs	Mesh with fault attr=3	No throw

3.1.6 Acceptance Criteria

- ☐ All 14 unit tests pass
- ☐ WaveOperator compiles, inherits DomainOperator<ParMesh>
- ☐ make test passes (all existing QD tests unaffected)

3.1.7 Dependencies

- Depends on: DomainOperator (existing), ConstitutiveModel (existing), BoundaryConfig (existing, extend)
- Required by: Phase 2, 3

3.2 Phase 2: Boundary Conditions — Absorbing + Free Surface

3.2.1 Goal

Wave operator handles absorbing (Eq. 5) and free-surface (Eq. 6) boundaries with verified reflection coefficients and energy absorption, using the same algorithms as SeisSol.

3.2.2 Files to Modify

- dynamic/wave_operator.cpp — BC dispatch in face flux loop
- dynamic/godunov_flux.cpp — Full implementation of Absorbing() and FreeSurface()

3.2.3 Files to Create

- tests/unit/test_wave_bc.cpp — 7 boundary condition tests

3.2.4 Detailed Requirements

3.2.4.1 1. Absorbing BC Implementation (Eq. 5)

```
void GodunovFlux::Absorbing(const real_t normal[3],
                           const real_t Q_self[9],
                           real_t flux[9]) const
{
    real_t t1[3], t2[3];
    BuildTangentFrame(normal, t1, t2);
    real_t Q_rot[9];
    RotateState(normal, t1, t2, Q_self, Q_rot); // T^{-1} Q
    real_t F_rot[9];
    Aplus_.Mult(Q_rot, F_rot); // A^+ Q_rot
    RotateStateInverse(normal, t1, t2, F_rot, flux); // T F_rot
}
```

3.2.4.2 2. Free Surface BC Implementation (Eq. 6)

```

void GodunovFlux::FreeSurface(const real_t normal[3],
                              const real_t Q_self[9],
                              real_t flux[9]) const
{
    // Rotate to face-normal frame
    real_t Q_rot[9];
    RotateState(normal, t1, t2, Q_self, Q_rot);

    // Apply Gamma mirror: flip sigma_{nn}, sigma_{nt1}, sigma_{nt2}
    real_t Q_ghost[9];
    Q_ghost[0] = -Q_rot[0]; // sigma_{nn}
    Q_ghost[1] = Q_rot[1]; // sigma_{t1 t1}
    Q_ghost[2] = Q_rot[2]; // sigma_{t2 t2}
    Q_ghost[3] = -Q_rot[3]; // sigma_{n t1}
    Q_ghost[4] = Q_rot[4]; // sigma_{t1 t2}
    Q_ghost[5] = -Q_rot[5]; // sigma_{n t2}
    for (int i = 6; i < 9; i++) Q_ghost[i] = Q_rot[i]; // velocities unchanged

    // Standard interior flux with ghost: A^+ Q_rot + A^- Q_ghost
    real_t F_rot[9], tmp[9];
    Aplus_.Mult(Q_rot, F_rot);
    Aminus_.Mult(Q_ghost, tmp);
    for (int i = 0; i < 9; i++) F_rot[i] += tmp[i];

    RotateStateInverse(normal, t1, t2, F_rot, flux);
}

```

3.2.5 Unit Tests for Phase 2

3.2.5.1 Test File: tests/unit/test_wave_bc.cpp

#	Test Name	What	How	Tolerance
15	TestAbsorbingNormalIncidence	P-wave exits with < 1% reflection	Long tube, Ricker pulse	$E_{\text{res}}/E_{\text{in}} < 0.01$
16	TestAbsorbingObliqueIncidence	Documents oblique reflection	30° incidence	$1\% < R < 20\%$ (regression)
17	TestFreeSurfacePReflection	P→P reflection: $R_{PP} = -1$ at normal incidence	Downward P-wave, measure reflected amplitude	$ R_{PP} + 1 < 0.05$
18	TestFreeSurfaceZeroTraction	$\sigma \cdot \mathbf{n} = \mathbf{0}$ at surface at all times	Random IC, monitor surface QPs	$\max \sigma \cdot \mathbf{n} < 10^{-10}$
19	TestFreeSurfaceEnergyConservation	Energy conserved with all free-surface BCs	All-free-surface box, point source	$ E_f/E_0 - 1 < 10^{-10}$

#	Test Name	What	How	Tolerance
20	TestMixedBCCorner	Corner element (free surface + absorbing) stable	100 steps, check no NaN	No NaN; energy decreasing
21	TestAbsorbingEnergyDecay	Energy monotonically decreasing	All-absorbing box	$E(t + \Delta t) \leq E(t) + \epsilon$

3.2.6 Acceptance Criteria

- ☐ All 7 BC tests pass (Tests 15-21)
- ☐ Phase 1 tests still pass
- ☐ make test still passes

3.2.7 Dependencies

- Depends on: Phase 1
- Required by: Phase 4

3.3 Phase 3: Fault-Face Riemann Solver Using Existing Friction Infrastructure

3.3.1 Goal

Dynamic rupture on a fault interface, with the Riemann solver reusing `DieterichRuinaFriction::SolveSlipRateVectorPsi()` for the friction solve (Eq. 8) and `FaultBasis` for coordinate transforms.

3.3.2 Files to Create

- `dynamic/fault_face_flux.hpp` — Trial traction + imposed state (~300 LOC)
- `dynamic/fault_face_flux.cpp` — Implementation
- `tests/unit/test_fault_face_flux.cpp` — 10 fault flux tests

3.3.3 Files to Modify

- `dynamic/wave_operator.cpp` — Integrate fault flux into face loop

3.3.4 Detailed Requirements

3.3.4.1 1. FaultFaceFlux Class

```
class FaultFaceFlux {
public:
    struct DOFData {
        real_t Zp_plus, Zp_minus, Zs_plus, Zs_minus;
        real_t eta_p, eta_s;
        real_t sigma_n0, tau1_0, tau2_0;
        real_t a, b, Dc, f0, V0;
        real_t psi, slip_rate, slip1, slip2;
    };

    FaultFaceFlux(DieterichRuinaFriction *friction,
                  const FaultBasis *basis,
                  real_t rho, real_t cp, real_t cs);
```

```

    /// Compute trial traction (Eq. 7a-c)
    void ComputeTrialTraction(const DOFData &data,
                             const real_t Q_plus[9], const real_t Q_minus[9],
                             real_t &sigma_n_trial,
                             real_t &tau1_trial, real_t &tau2_trial) const;

    /// Full fault flux: Eq. 7  $\rightarrow$  8  $\rightarrow$  9  $\rightarrow$  10  $\rightarrow$  11/12  $\rightarrow$  1
    void Evaluate(int fi, int qi, DOFData &data,
                 const real_t Q_plus[9], const real_t Q_minus[9],
                 real_t dt,
                 real_t imposed_plus[9], real_t imposed_minus[9]) const;

    /// Analytic state update (Eq. 13)
    static real_t UpdateStateAnalytic(real_t psi_old, real_t V, real_t Dc, real_t dt);

private:
    DieterichRuinaFriction *friction_;
    const FaultBasis *basis_;
    real_t rho_, cp_, cs_, Zp_, Zs_;
};

```

3.3.4.2 2. Trial Traction (Eq. 7) Implements Eq. (7a-c) matching SeisSol FrictionSolverCommon.h:182-192:

```

sigma_n_trial = data.eta_p * (v_n_minus - v_n_plus
                             + sigma_n_plus / data.Zp_plus
                             + sigma_n_minus / data.Zp_minus);

```

3.3.4.3 3. Full Evaluate (Eq. 7-13)

```

void FaultFaceFlux::Evaluate(...) const
{
    /// Step 1: Trial traction (Eq. 7)
    ComputeTrialTraction(data, Q_plus, Q_minus, sn_trial, t1_trial, t2_trial);

    /// Step 2: Total traction
    real_t tau_total[2] = { data.tau1_0 + t1_trial, data.tau2_0 + t2_trial };
    real_t sigma_n_total = max(data.sigma_n0 + sn_trial, 0.0);

    /// Step 3: Friction solve (Eq. 8) - reuses existing Brent solver
    real_t V_vec[2];
    friction_>SolveSlipRateVectorPsi(tau_total, data.psi,
                                     sigma_n_total, data.eta_s, data.a, V_vec);

    /// Step 4: Corrected traction (Eq. 10)
    real_t t1_corr = t1_trial - data.eta_s * V_vec[0];
    real_t t2_corr = t2_trial - data.eta_s * V_vec[1];

    /// Step 5: Imposed state (Eq. 11-12)
    /// Plus side: v_imp = v + (1/Z)(tau_corr - tau)
    /// Minus side: v_imp = v - (1/Z)(tau_corr - tau)

    /// Step 6: State update (Eq. 13)
    data.psi = UpdateStateAnalytic(data.psi, V_mag, data.Dc, dt);
}

```

```
}
```

3.3.4.4 4. State Variable Update (Eq. 13)

```
real_t FaultFaceFlux::UpdateStateAnalytic(real_t psi_old, real_t V, real_t Dc, real_t dt)
{
    if (V < 1e-30) return psi_old + dt;
    real_t x = -V * dt / Dc;
    return psi_old * std::exp(x) + (Dc / V) * (-std::expm1(x));
}
```

3.3.5 Unit Tests for Phase 3

3.3.5.1 Test File: tests/unit/test_fault_face_flux.cpp

#	Test Name	What	Tolerance
22	TestTrialTractionLockedFault	$\mathbf{Q}^+ = \mathbf{Q}^- \Rightarrow$ trial = background traction	$< 10^{-10}$
23	TestTrialTractionVelocityJump	Pure $\Delta v_{t_1} = 1$: $\tau_1^{\text{trial}} = -\eta_s$	$< 10^{-12}$
24	TestLockedFaultZeroSlip	High friction ($a = 0.5$) $\Rightarrow V \approx 0$	$V < 10^{-20}$
25	TestFrictionlessFaultFullDrop	Zero friction $\Rightarrow$ $\hat{V} = \Theta/\eta_s$	$< 10^{-10}$
26	TestImposedStateContinuity	$\sigma^{+, \text{imp}} = \sigma^{-, \text{imp}}$ (traction continuous)	$< 10^{-14}$
27	TestImposedStateVelocityJump	$v_{t_1}^{+, \text{imp}} - v_{t_1}^{-, \text{imp}} = V_1$	$< 10^{-12}$
28	TestSolverReuse	Same V from standalone call and from Evaluate()	$< 10^{-10}$
29	TestStateVariableAnalytic	ψ after 1000 steps matches ψ_{exact} for constant V	$< 10^{-10}$
30	TestEnergyBalance	$\Delta E_{\text{domain}} + \int \tau \cdot V dA dt = 0$	$< 10^{-8} E_0$
31	TestFaultFluxSignConvention	Positive $V \Rightarrow$ $\tau^{\text{corr}} < \tau^{\text{trial}}$	Qualitative

3.3.6 Acceptance Criteria

- ☐ All 10 fault flux tests pass (Tests 22–31)
- ☐ Phase 1 and 2 tests still pass

3.3.7 Dependencies

- Depends on: Phase 1, DieterichRuinaFriction, FaultBasis, AgingLawPsi
- Required by: Phase 4

3.4 Phase 4: TPV102 Full Benchmark

3.4.1 Goal

Complete TPV102 simulation using SeisSol-compatible mesh and parameters, producing SCEC-format output, verified against community median.

3.4.2 TPV102 Parameters

Parameter	Symbol	Value
Density	ρ	2670 kg/m ³
S-wave speed	c_s	3464 m/s
P-wave speed	c_p	6000 m/s
Shear modulus	$\mu = \rho c_s^2$	32.04 GPa
First Lamé	$\lambda = \rho c_p^2 - 2\mu$	32.04 GPa
Reference friction	f_0	0.6
Reference velocity	V_0	10 ⁻⁶ m/s
State evolution	b	0.012
Characteristic slip distance	L	0.02 m
Direct effect (VW)	a_{vw}	0.008 ($a - b = -0.004$)
Direct effect (VS)	a_{vs}	0.016 ($a - b = +0.004$)
Normal stress	σ_n	120 MPa (compressive)
Initial shear traction	τ_{ini}	75 MPa
Initial slip rate	V_{ini}	10 ⁻¹² m/s
VW zone half-width	W	15 km
Transition width	w	3 km
Nucleation stress	$\Delta\tau_0$	25 MPa
Nucleation radius	R	3 km
Nucleation rise time	T	1 s
Simulation time	t_f	12 s

3.4.3 Friction Law

Regularized aging law rate-and-state:

$$\tau = a \sigma_n \operatorname{arcsinh} \left[\frac{V}{2V_0} \exp \left(\frac{f_0 + b \ln(V_0 \theta / L)}{a} \right) \right]$$

$$\frac{d\theta}{dt} = 1 - \frac{V\theta}{L}$$

3.4.4 Nucleation

$$\Delta\tau(x, z, t) = \Delta\tau_0 \cdot F(r) \cdot G(t)$$

$$F(r) = \begin{cases} \exp\left(\frac{r^2}{r^2 - R^2}\right) & r < R \\ 0 & r \geq R \end{cases}, \quad G(t) = \begin{cases} \exp\left(\frac{(t - T)^2}{t(t - 2T)}\right) & 0 < t < T \\ 1 & t \geq T \end{cases}$$

3.4.5 Initial θ from Self-Consistency

$$\theta_{ini} = \frac{L}{V_{ini}} \exp \left(\frac{a \operatorname{arcsinh} \left(\frac{V_{ini}}{2V_0} e^{f_0/a} \right) - \tau_{ini}/\sigma_n}{b} \right)$$

3.4.6 Files to Create

- config/tpv102_params.hpp — Parameter struct
- config/tpv102.toml — TOML configuration
- dynamic/tpv102_setup.hpp/.cpp — Spatial params, nucleation, stations
- drivers/tpv102_driver.cpp — Main driver
- tpv102/mesh/tpv102.geo — Gmsh geometry (matching SeisSol mesh)
- tests/unit/test_tpv102_setup.cpp — 5 setup tests
- tests/verification/tpv102_verification.py — SCEC comparison

3.4.7 Unit Tests for Phase 4

#	Test Name	What	Tolerance
32	TestBoxcarFunction	$B(0, W, w) = 1$, $B(2W, W, w) = 0$, smooth transition	Exact interior/exterior
33	TestSpatiallyVaryingA	$a(0, -7.5\text{km}) = 0.008$, $a(25\text{km}, -25\text{km}) = 0.024$	$< 10^{-12}$
34	TestInitialStateEquilibrium	$\tau_{\text{ini}} = \sigma_n f(V_{\text{ini}}, \psi_{\text{ini}}) + \eta V_{\text{ini}}$	$ \text{res} /\tau < 10^{-10}$
35	TestNucleationPerturbation	$\Delta\tau = 0$ at $r > R$; $\Delta\tau = \Delta\tau_0$ at $t > T, r = 0$	$< 10^{-10}$
36	TestStationDefinitions	9 fault + 6 surface stations at SCEC coordinates	Exact match

3.4.8 Acceptance Criteria

- ☐ All 5 setup tests pass
- ☐ Rupture nucleates at hypocenter within $t < 2$ s
- ☐ Peak slip rate at station $(0, -7.5 \text{ km})$ within 10% of community median
- ☐ Rupture arrival time within 5% of community median
- ☐ Simulation completes 12 s without instability
- ☐ All previous tests pass

3.4.9 Dependencies

- Depends on: Phase 1, 2, 3
- Required by: Phase 5

3.5 Phase 5: BP5 QD→Dynamic Transfer

3.5.1 Goal

Enable transition from quasi-dynamic BP5 to dynamic rupture during seismic events, and back to QD after event arrest. Connects existing BP5 infrastructure to the new dynamic solver.

3.5.2 Activation/Deactivation Criteria

$$\text{QD} \rightarrow \text{FD} : \max_{\text{fault}}(V) > V_{\text{activate}} \quad \text{and} \quad \frac{d}{dt} \max(V) > 0$$

$$\text{FD} \rightarrow \text{QD} : \max_{\text{fault}}(V) < V_{\text{deactivate}} \quad \text{for } \geq 10 \text{ consecutive steps}$$

3.5.3 State Conversion: QD $\rightarrow$ FD

1. Compute stress from displacement: $\sigma = \mathbb{C} : \varepsilon = \mathbb{C} : \text{sym}(\nabla \mathbf{u})$
2. Set velocity from QD slip rate: $\mathbf{v} = \pm V_{\text{qd}}/2 \cdot \hat{\mathbf{s}}$ at fault, $\mathbf{v} = \mathbf{0}$ in bulk
3. Add tectonic background: $\sigma_{\text{total}} = \sigma_{\text{from u}} + \sigma_0$

3.5.4 State Conversion: FD $\rightarrow$ QD

1. Accumulate slip: $\delta_{\text{new}} = \delta_{\text{frozen}} + \int_{\text{FD}} V dt$
2. Transfer ψ directly
3. Solve QD elasticity with new slip: $\mathbf{K} \mathbf{u}_{\text{new}} = \mathbf{f}(\delta_{\text{new}})$

3.5.5 Files to Create

- dynamic/regime_transfer.hpp/.cpp — Conversion utilities (~400 LOC)
- solver/seas_hybrid_operator.hpp/.cpp — Three-mode dispatcher (~300 LOC)
- tests/unit/test_regime_transfer.cpp — 6 tests

3.5.6 Unit Tests for Phase 5

#	Test Name	What	Tolerance
37	TestQDtoFDStressConsistency	$\ \sigma_{\text{QD}} - \sigma_{\text{FD}}\ _{L^2} / \ \sigma_{\text{QD}}\ _{L^2}$ small	$< 10^{-6}$
38	TestQDtoFDSlipRateContinuity	$V_{\text{FD}} = V_{\text{QD}}$ at fault	$\ V_{\text{FD}} - V_{\text{QD}}\ _{\infty} < 10^{-10}$
39	TestFDtoQDRoundTrip	QD $\rightarrow$ FD $\rightarrow$ QD (zero evolution) = identity	$< 10^{-12}$
40	TestFDtoQDSlipAccumulation	Known $\Delta\delta$ during FD transferred correctly	$< 10^{-12}$
41	TestFDtoQDPsiPreservation	ψ preserved across FD $\rightarrow$ QD	$< 10^{-14}$
42	TestHybridOperatorSwitching	Correct regime transitions at thresholds	Exact

3.5.7 Acceptance Criteria

- ☐ All 6 transfer tests pass (Tests 37–42)
- ☐ Round-trip preserves state to machine precision
- ☐ All previous tests pass

3.5.8 Dependencies

- Depends on: Phase 4, existing QD infrastructure
- Required by: Future hybrid BP5 simulations

3.6 Phase 6: GPU Acceleration

3.6.1 Goal

Accelerate the dynamic rupture solver using MFEM's GPU infrastructure (MFEM_FORALL, Device, Vector::UseDevice). Target: $\geq 5\times$ speedup over single-core CPU for TPV102.

3.6.2 MFEM GPU Infrastructure

Component	File	Key API
Device abstraction	general/device.hpp	Device::Configure("cuda")
Kernel launch	general/forall.hpp	MFEM_FORALL(i, N, {...})
Memory management	general/mem_manager.hpp	Vector::UseDevice(true), .Read(), .Write()
DG face restriction	fem/restriction.hpp	L2FaceRestriction(scatter/gather)
PA bilinear form	fem/bilinearform.hpp	SetAssemblyLevel(AssemblyLevel::PARTIAL)
DG diffusion PA	fem/integ/bilininteg_dgd	Face integration patterns

3.6.3 Task Breakdown

Task	Description	Deliverable
6.1	CPU profiling baseline (gprof/perf)	Percentage breakdown: volume, face, fault, $\mathbf{M}^{-1}$, MPI
6.2	AoS→SoA data layout via ElementRestriction	Q_E[comp][dof][elem] coalesced format
6.3	GPU volume integral kernel	MFEM_FORALL(e, ne, {...}) — one block per element
6.4	GPU face flux kernel	MFEM_FORALL(f, nf, {...}) — one thread per face
6.5	GPU mass inverse kernel	MFEM_FORALL(e, ne, {...}) — embarrassingly parallel
6.6	GPU fault friction kernel	Fixed 15-iteration Brent (mitigates thread divergence)
6.7	Memory management	UseDevice(true), precompute $\mathbf{T}$, $\mathbf{T}^{-1}$, $\mathbf{A}^{\pm}$ on device

3.6.4 GPU Volume Integral Kernel (Task 6.3)

Implements Eq. (2b) on GPU:

$$\text{rhs}(c, i, e) += w_q \sum_{j=1}^3 G(i, q, j) [\mathbf{A}_j \mathbf{Q}(\mathbf{x}_q)]^c$$

```
MFEM_FORALL(e, ne, {
  // Evaluate Q at all QPs: Q_qp[q][c] = sum_i B(i,q) * Q_E(c,i,e)
```

```

// Compute flux: F_j[c] = A_j * Q_qp for j=x,y,z
// Accumulate: rhs(c,i,e) += w * sum_j G(i,q,j) * F_j[c]
});

```

3.6.5 GPU Fault Friction Kernel (Task 6.6)

Challenge: Brent's method has variable iteration count $\Rightarrow$ thread divergence.

Mitigation: Fixed 15-iteration loop with early-exit flag. Alternative: Newton-Raphson on GPU (matches SeisSol GPU approach) with Brent fallback on CPU.

3.6.6 Files to Create

- dynamic/wave_kernels_gpu.hpp/.cpp — GPU kernels (~600 LOC)
- tests/unit/test_wave_gpu.cpp — 4 GPU equivalence tests

3.6.7 Unit Tests for Phase 6

#	Test Name	What	Tolerance
43	TestVolumeIntegralGPUvsCPU	GPU volume RHS = CPU volume RHS	$< 10^{-12}$
44	TestFaceFluxGPUvsCPU	GPU face flux = CPU face flux	$< 10^{-12}$
45	TestFullMultGPUvsCPU	Mult() GPU = CPU	$\ \cdot \ _{\infty} / \ \cdot \ _{\infty} < 10^{-11}$
46	TestGPUEnergyConservation	Reflecting box on GPU preserves energy	$ E_f/E_0 - 1 < 10^{-9}$

3.6.8 Acceptance Criteria

- ☐ CPU profiling report completed
- ☐ All 4 GPU tests pass (Tests 43–46)
- ☐ $\geq 5\times$ speedup on NVIDIA A100 or similar
- ☐ Graceful fallback with Device("cpu")
- ☐ All previous tests pass

3.6.9 Dependencies

- Depends on: Phase 4 (working CPU code)
- Required by: nothing (optimization)

4 PART III: CROSS-CUTTING CONCERNS

4.1 Testing Strategy Summary

Phase	Test File	# Tests	Verifies
1	test_godunov_flux.cpp	8	Jacobians, eigenvalues, rotation, flux consistency

Phase	Test File	# Tests	Verifies
1	test_wave_operator.cpp	6	Mass inverse, plane waves, convergence, energy, fault geometry
2	test_wave_bc.cpp	7	Absorbing/free-surface reflection, energy, mixed BCs
3	test_fault_face_flux.cpp	10	Trial traction, locked/frictionless, continuity, energy balance
4	test_tpv102_setup.cpp	5	Boxcar, spatial params, equilibrium, nucleation, stations
5	test_regime_transfer.cpp	6	Stress consistency, round-trip, slip accumulation, switching
6	test_wave_gpu.cpp	4	GPU vs CPU equivalence for all kernels
Total		46	

4.2 Risk Assessment

Risk	Likelihood	Impact	Mitigation
η_{QD} vs η_s mismatch	Medium	High	Test 28: Compare V from Riemann call vs standalone. For homogeneous: $\eta_{\text{QD}} = \eta_s$ exactly.
Sign convention mismatch at fault	Medium	High	Test 31: Locked fault $\Rightarrow V = 0$; wrong sign $\Rightarrow$ immediate blowup.
Free surface Γ wrong indices	Medium	High	Test 18: Zero-traction check every step. Test 7: Ghost state verification.
L^2 9-component memory overhead	Medium	Medium	Profile in Task 6.1. Consider separate scalar spaces if $> 2\times$.
GPU thread divergence in Brent	High	Medium	Fixed iteration count (15). Newton option for GPU.
QD $\rightarrow$ FD stress projection accuracy	Medium	Medium	Test 37: Relative L^2 error $< 10^{-6}$.
Absorbing BC reflections contaminate fault	Low	Medium	Domain large enough (60 km). Test 16 documents $R(\theta)$.

4.3 Project Layout After All Phases

```

miniapps/seas/
├─ dynamic/
|   └─ wave_state.hpp
NEW (Phases 1–6)
Phase 1: Q index enum, energy utilities

```

		— wave_operator.hpp	Phase 1: inherits DomainOperator
		— wave_operator.cpp	Phase 1: volume + face assembly
		— godunov_flux.hpp	Phase 1: upwind flux (rotation, split, BCs)
		— godunov_flux.cpp	Phase 1: implementation
		— fault_face_flux.hpp	Phase 3: trial traction + imposed state
		— fault_face_flux.cpp	Phase 3: reuses DieterichRuinaFriction
		— tpv102_setup.hpp	Phase 4: spatial params, nucleation, stations
		— tpv102_setup.cpp	Phase 4: implementation
		— regime_transfer.hpp	Phase 5: QD $\rightarrow$ FD conversion
		— regime_transfer.cpp	Phase 5: implementation
		— wave_kernels_gpu.hpp	Phase 6: MFEM_FORALL kernels
		— wave_kernels_gpu.cpp	Phase 6: GPU implementations
		— config/	
		— tpv102_params.hpp	Phase 4: benchmark parameters
		— tpv102.toml	Phase 4: TOML configuration
		— domain/	
		— boundary_config.hpp	Phase 1: +absorbing_attrs field
		— domain_config.hpp	Phase 1: +cfl_factor field
		— drivers/	
		— tpv102_driver.cpp	Phase 4: standalone dynamic driver
		— solver/	
		— seas_hybrid_operator.hpp	Phase 5: three-mode dispatcher
		— seas_hybrid_operator.cpp	Phase 5: implementation
		— tpv102/	
		— mesh/	
		— tpv102.geo	Phase 4: Gmsh geometry script
		— tests/	
		— unit/	
		— test_godunov_flux.cpp	Phase 1: 8 flux tests
		— test_wave_operator.cpp	Phase 1: 6 operator tests
		— test_wave_bc.cpp	Phase 2: 7 BC tests
		— test_fault_face_flux.cpp	Phase 3: 10 fault flux tests
		— test_tpv102_setup.cpp	Phase 4: 5 setup tests
		— test_regime_transfer.cpp	Phase 5: 6 transfer tests
		— test_wave_gpu.cpp	Phase 6: 4 GPU tests
		— verification/	
		— tpv102_verification.py	Phase 4: SCEC comparison script

New files: ~22. New LOC estimate: ~4,000 (including ~1,500 LOC of unit tests). **Existing code reused unchanged: ~8,000 LOC** (friction, fault basis, config, I/O). **Total unit tests: 46 new** (on top of existing QD test suite).